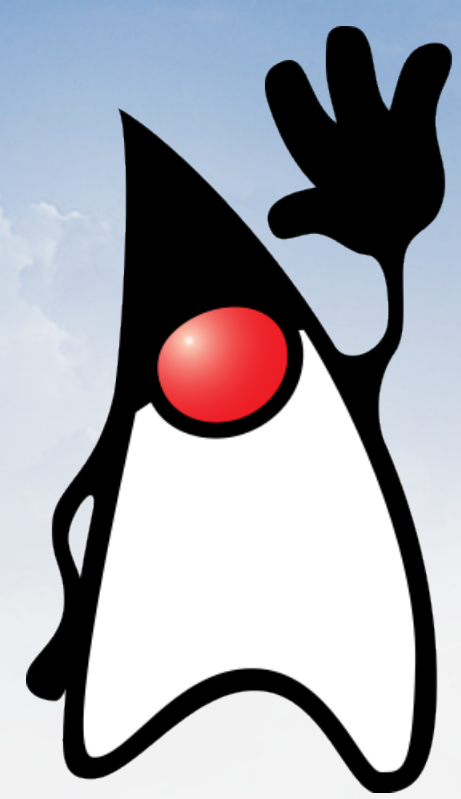




Interfaces

COP 3330

Spring 2024

Department of Computer Science

University of Central Florida

Dr. Bacanli



How to solve non multiple inheritance?

- Alligator lives on land and under water
 - We cannot extend landAnimal and waterAnimal at the same time
 - However, we can create waterBreathable interface
- An interface is like a bag of methods/abilities
 - If you want those abilities, implement the interface
- If that gift is given to you (Breathing under water)
 - You will need to override methods inside
 - You will need to know how to breathe under water



Interface example

```
package animals;
public interface waterBreathable{
    public void breatheUnderWater();
}
```

```
package animals;
public class Alligator extends LandAnimal
implements waterBreathable{

    public Alligator(){
        super();
    }
    public void breatheUnderWater(){
        System.out.println("Alligator under water");
    }
    public static void main(String[] args){
        Alligator ally=new Alligator();
        ally.breatheUnderWater();
        ally.walk();
    }
}
```

```
package animals;
public class Animal{

    public Animal(){
    }

    public void walk(){
        System.out.println("Animal walking");
    }
}
```

```
package animals;
public class LandAnimal extends Animal{

    public LandAnimal(){
        super();
    }
    @Override
    public void walk(){
        System.out.println("LandAnimal
walking");
    }
}
```

What is an interface?

- Cay Horstmann describes it as “A way of describing what classes should do.” from the Java Volume 1 Fundamentals book.
- A set of requirements that a class must implement.
- It is not a class!
- All methods defined in the interface are automatically *public* and *abstract*.

Interesting Properties of Interfaces

- Interfaces are NOT classes

`WaterBreathable w1 = new WaterBreathable();` INVALID

- Interface variables exist

`WaterBreathable w1; //possible`

- The interface variable **MUST** reference an object of the class that implements the interface.

Interfaces VS Abstract Classes

- People often confuse when to use interfaces over an abstract class or the other way around.
- Interfaces should be used instead of an abstract class when there is no default implementation to inherit.
- Classes can implement many interfaces
 - but extend only one class
- Interfaces do NOT have constructors, no object can be created from that

Interface methods and variables

- Interface methods are *public abstract*
- Interface variables are *public final static*

public: for the accessibility across all the classes, just like the methods present in the interface

static: as interface cannot have an object, the `interfaceName.variableName` can be used to reference it or directly the `variableName` in the class implementing it.

final: to make them constants. If 2 classes implement the same interface and you give both of them the right to change the value, conflict will occur in the current value of the var, which is why only one time initialization is permitted. (in order to avoid the deadly diamond for multiple inheritance)

Also all these modifiers are implicit for an interface, you don't really need to specify any of them.

One more point

- Since the methods are by default abstract
 - Implementing a method in an interface is not suggested
 - However, it is possible

```
public interface WaterBreathable{  
    int number=45; // You must initialize  
    public default int givezero() {  
        return 0;  
    }  
    //You won't need default keyword in this  
course  
}
```

Implementing an interface

- In order to implement an interface, the keyword “implements” must be used in the class definition
 - You will need to add commas between interfaces
 - Write extends first then implements!

```
public class Alligator extends LandAnimal implements WaterBreathable,  
AirBreathable, WaterSwimmer
```

Real life example where Interfaces are used

Comparable<T> interface
and
compareTo(T Obj) method

More Properties

- You can use the instanceof operator to check whether an object implements an interface

```
if (myObj instanceof Comparable)
{
    ...
}
```

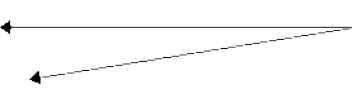
- You can even extend interfaces!

```
public interface Vehicle extends Hover
{
    double milesPerGallon();
    double SPEED_LIMIT = 100; //public static final constant
}
```

The Comparable Interface

- One common interface Java developers use is the comparable interface.
- Why?
 - Let's say you want to sort an Array, LinkedList, or some data structure that contains a class object with multiple attributes.
 - How would we specify which attribute to use?

```
public class Employee implements Comparable<Employee>
{
    private String name;
    private double salary;
```



Which one?

The compareTo Method

- Once the class implements the interface, it is required that a definition of the method is provided in the class.
- If you try to sort some collection without compareTo definition
 - Java will give compilation error

```
public int compareTo(Employee other)
{
    return Double.compare(salary, other.salary);
}
```

```
package Employees;
public abstract class Employee implements
Comparable<Employee>{
    public int salary;

    public Employee(int salary){
        this.salary=salary;
    }
    //Either this method or the next one!
    public int compareTo(Employee e){
        return Integer.compare(salary,e.salary);
    }

    @Override
    public int compareTo(Employee e) {
        if (salary < e.salary) {
            return -1; //any negative number
        }else if (salary == e.salary()) {
            return 0;
        }
        return 1;//any positive number
    }
}
```

Comparable

```
package Employees;
import java.util.*;
public class EmployeesTester{
    public static void main(String[] args){
        ArrayList<Employee> arr;
        arr=new ArrayList<>();
        arr.add(new Employee(100));
        arr.add(new Employee(500));
        arr.add(new Employee(50));
        System.out.println(arr);
        Collections.sort(arr);
        System.out.println(arr);
    }
}
```

What is the output?

Why did the java developers create an interface but not a comparable class?

Returning/Taking Interface type

```
WaterBreathable returnInterfaceType(Alligator a){  
    return a;  
}
```

```
LandAnimal returnSuperType(Alligator a){  
    return a;  
}
```

```
void wrapperMethod(WaterBreathable animal){  
    animal.breatheUnderWater(); // You can only call methods inside WaterBreathable  
    // not all animals may be able to walk but  
    // they all can breatheUnderWater()  
}
```